

Collections API – Business Connect Specification (v2.0)

Contents

1. Purpose 3
2. Authentication Implementation Guide 3
   2.1 JSON Web Token (JWT) 3
   2.2 OAuth 2.0 (Client Credentials) 4
   2.3 API Key 4
   2.4 Basic Authentication 4
3. Validation API 5
   3.1 Endpoint 5
   3.2 Request Specification 5
   3.3 Example Responses 5
   3.4 Response Specification 6
4. Payments Request Specification 6
   4.1 Unified Request Payload Example 6
4.2 Request Field Specification 7
4.3 Payments Response Specification 8
   4.4 Example Responses 8

1. Purpose

The Collections API is designed to receive and process payments made by customers through various channels. It ensures all payment types follow a standard format so they can be processed quickly and accurately.

2. Authentication Implementation Guide

The Collections API requires secure and reliable authentication to ensure that only authorized systems and users can access its services. This section outlines the supported authentication methods, the required request headers, and the recommended best practices for implementation.

2.1 JSON Web Token (JWT)

JWT-based authentication is used when a client authenticates using credentials (e.g., username and password) to obtain a token.

- **Login / Token Retrieval**
  - **HTTP Method: POST**
  - **Content-Type: application/json**

  Example Request:

  ```
  POST /auth/login
  Host: api.example.com
  Content-Type: application/json

  {
    "username": "john.doe",
    "password": "mySecret123"
  }
  ```

  Example Response:

  ```
  {
    "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6...",
    "expires_in": 3600
  }
  ```

- **Use Token in Requests**
- `POST /api/send HTTP/1.1`
- `Host: api.example.com`
- `Authorization: Bearer eyJhbGciOiJIUzI1NiIs...`

2.2 OAuth 2.0 (Client Credentials)

The OAuth 2.0 client credentials flow is recommended for server-to-server integrations where no user interaction is required.

- **Token Retrieval**
  - **HTTP Method:** POST
  - **Content-Type:** application/x-www-form-urlencoded
- **Example Request:**
- `POST /oauth/token`
- `Host: api.example.com`
- `Content-Type: application/x-www-form-urlencoded`
- `grant_type=client_credentials&client_id=myClientId&client_secret=mySecretKey`

- **Example Response:**
- `{`
- `  "access_token": "ya29.a0AfH6SMD...",`
- `  "expires_in": 3600`
- `}`
- **Use Token in Requests**
- `POST /api/data HTTP/1.1`
- `Host: api.example.com`
- `Authorization: Bearer ya29.a0AfH6SMD...`

2.3 API Key

A simple authentication method where an API key is issued to a client and sent with each request.

- **Example Request with Header:**
- `POST /api/data`
- `Host: api.example.com`
- `x-api-key: 12345ABCDE`

2.4 Basic Authentication

The most straightforward method where the client sends a username and password in the `Authorization` header encoded with Base64.

- **Formula for Header Value:**
  `Authorization: Basic base64(username:password)`
- **Example Request:**
- `POST /api/data HTTP/1.1`
- `Host: api.example.com`
  `Authorization: Basic am9obi5kb2U6bXlTZWNyZXQxMjM=`

3. Validation API

The Validation API is designed to **verify the validity of customer references before a payment request is initiated**. By confirming account details early in the transaction process, the API helps ensure that payments are directed to valid and active customer accounts.

3.1 Endpoint

```
POST /api/v1/example/validate
```

3.2 Request Specification

Field	Type	Required	Description
customerRef	String	Yes	Unique customer reference identifier to validate

Example Request:

```
{
  "customerRef": "C1012"
}
```

3.3 Example Responses

✅ Successful Validation (200 OK)

```
{
  "resultCode": 0,
  "message": "Customer found",
  "customerName": "Clark Kent",
  "customerRef": "B101"
}
```

❌ Invalid Customer Reference (404 Not Found)

```
{
  "resultCode": 1,
  "message": "Customer not found",
  "customerName": "",
  "customerRef": ""
}
```

🔒 Unauthorized (401 Unauthorized)

```
{
  "resultCode": 1,
  "message": "Authentication failed. Token missing or expired.",
```

```
  "customerName": "",
  "customerRef": ""
}
```

💥 **Internal Server Error (500 Internal Server Error)**

```
{
  "resultCode": 1,
  "message": "An unexpected error occurred while processing the request.
Please try again later.",
  "customerName": "",
  "customerRef": ""
}
```

3.4 Response Specification

Field	Type	Description
resultCode	Int	Indicates the outcome of the request. `0` = success, `1` = failure.
message	String	Human-readable description of the result (e.g., “Customer found”).
customerName	String	Name of the customer if validation is successful, otherwise an empty string
customerRef	String	Unique identifier of the customer if found, otherwise an empty string

4. Payments Request Specification

All payment requests must be submitted in **JSON format**. The request follows a unified payload structure, with **conditional fields** inside the `additions` object that vary depending on the payment type.

> **Note:** The Payments API supports the following channels:
>
> - **M-PESA** (Paybill, Till)
> - **RTGS** (Real-Time Gross Settlement transfers)
> - **PesaLink** (interbank instant payments)
> - **OTG** (Internet/Mobile payments)
> - **Branch** (in-branch initiated transactions)
> - **Cheque** (physical cheque deposits/payments)

4.1 Unified Request Payload Example

```
{
  "paymentType": "CHEQUE",
  "transactionReference": "S44998347",
```

```
  "transactionDate": "2025-08-15T10:30:00",
  "amount": 2500.00,
  "currency": "KES",
  "shortCode": "123456",
  "additions": {
    "customerRef": "TESTUERS",             // Required for all payment types
except M-PESA Paybill (no customer ref) and M-PESA Till
    "externalRefNumber": "CHQ1234567",     // Required for all payment types
(e.g., M-PESA Ref No., Cheque deposit slip no.)
    "payerName": "John Doe",               // Required for M-PESA and Cheque
payments
    "payerMobileNumber": "254712345678",   // Required for M-PESA
Paybill/Till types
    "senderBankCode": "B101",              // Required for Bank payment types
(RTGS, PesaLink, OTG)
    "senderAccountNumber": "930393039124", // Required for Bank and Cheque
payments
    "senderAccountName": "Jane Doe",       // Required for Bank and Cheque
payments
    "senderAddress": "Nairobi",            // Optional for Bank payments
    "chequeNumber": "987654"               // Required for Cheque payments
  }
}
```

4.2 Request Field Specification

Field	Type	Required	Description
paymentType	String	Yes	The payment channel type. Supported values: `MPESA`, `RTGS`, `PESALINK`, `OTG`, `BRANCH`, `CHEQUE`.
transactionReference	String	Yes	Unique transaction reference number generated by the client system.
transactionDate	DateTime	Yes	The date and time of the transaction in ISO 8601 format.
amount	Number	Yes	The transaction amount.
currency	String	Yes	Currency code (ISO 4217), e.g., `KES`, `USD`.
shortCode	String	Yes	The shortcode or merchant identifier receiving the payment.
additions	Object	Conditional	Additional fields that vary by payment type.
→ customerRef	String	Conditional	Required for all payment types **except** M-PESA Paybill (without reference) and M-PESA Till.

→ externalRefNumber	String	Yes	Always required. Represents the provider reference (e.g., M-PESA Ref No., Cheque deposit slip).
→ payerName	String	Conditional	Required for **M-PESA** and **Cheque** payments.
→ payerMobileNumber	String	Conditional	Required for **M-PESA Paybill/Till** payments.
→ senderBankCode	String	Conditional	Required for **Bank payments** (`RTGS`, `PesaLink`, `OTG`).
→ senderAccountNumber	String	Conditional	Required for **Bank** and **Cheque** payments.
→ senderAccountName	String	Conditional	Required for **Bank** and **Cheque** payments.
→ senderAddress	String	Optional	Optional for **Bank payments**.
→ chequeNumber	String	Conditional	Required for **Cheque** payments.

4.3 Payments Response Specification

Field	Type	Description
resultCode	Number	Indicates the outcome of the request. `0` = success, `1` = failure.
resultDesc	String	Human-readable description of the payment result.
erpRefId	String	Unique reference ID generated by the ERP system for tracking the payment.

4.4 Example Responses

✅ Successful Payment (HTTP 200/201)

```
{
  "resultCode": 0,
  "resultDesc": "Payment received successfully",
  "erpRefId": "ERP98765"
}
```

❌ Client Error (HTTP 400 – Invalid Payload)

```
{
  "resultCode": 1,
  "resultDesc": "Missing required field: transaction reference",
  "erpRefId": null
```

```
}
```

🔒 Unauthorized (HTTP 401 – Authentication Failed)

```
{
  "resultCode": 1,
  "resultDesc": "Authentication failed. Token missing, invalid, or expired.",
  "erpRefId": null
}
```

💥 Server Error (HTTP 500 – Processing Failure)

```
{
  "resultCode": 1,
  "resultDesc": "Internal server error, please try again later",
  "erpRefId": null
}
```